Antonio Conopio

647-807-8795 | aconopio@uoguelph.ca | linkedin | github | website

Education

Bachelor of Computing, Software Engineering (Co-op), Minor in Math Sep 2023 – Apr 2028
University of Guelph *Guelph, ON*

- **Cumulative GPA:** 3.89/4.0 (88.4%) | Dean's List: 2023 – present
- **Relevant Coursework:** Data Structures & Algorithms, Operating Systems, Parallel Programming, Software Testing

Experience

Data Integration Intern (API Developer) Sep 2025 – Aug 2026
Co-operators *Guelph, ON*

- Migrated the team's API template from R/Plumber to Python/FastAPI, cutting average response latency by 33% and raising test coverage from 40% to 97%; validated parity with the legacy version before team-wide adoption.
- Gathered requirements from business partners and extended the auto and home Risk Assist APIs with new fields, validation, and business rules, enabling two new business channels and supporting a 30% increase in API call volume.
- Led testing for the National Auto Risk Assist API, a flagship project, debugging across multiple models and data transformations to reach 100% parity with the modelers' pipeline before release.
- Isolated a reconciliation defect blocking a project by using batch testing to narrow it to a specific pipeline stage, then shipped the fix.
- Built and deployed an R/Plumber API on Posit to extract historical API logs, plus an automation script that calls it to migrate 1M+ logs to Azure Data Lake Storage (ADLS) for the cloud modernization initiative.
- Built a comparator script that automatically diffs expected vs. actual API responses to speed up regression testing, and supported 10+ production deployments, including validation and rollback planning.

Projects

Orion | *Go, Python, TypeScript, React (Next.js), Redux Toolkit, PostgreSQL, Redis, WebSockets, Docker*

- Built a distributed DAG workflow orchestrator end-to-end (Go REST API, Python worker pool, Next.js console) with race-safe fan-in joins that skip downstream tasks on failure, verified by 230 tests against real Postgres and Redis.
- Engineered a Redis Streams consumer-group queue with at-least-once delivery, idempotent task starts that drop duplicates, and crash recovery (stale-message reclaim plus a re-queue sweeper) that restarts a killed worker's task in ∼150 s.
- Designed task-to-task data passing: tasks publish JSON outputs that downstream tasks read, persisted in the same Postgres transaction as task status and validated so a bad result fails its task instead of corrupting downstream work.
- Built a Go WebSocket hub (Redis Pub/Sub fan-out) that pushes run and worker updates to the React frontend instead of polling, delivering status changes ∼14 ms after the database commit (p95 16 ms).
- Containerized the stack with Docker Compose (6 services, one-command startup, auto-applied migrations); throughput scaled near-linearly with worker count, reaching 5.5× with 6 workers on 0.5 s tasks (1.9 → 10.3 tasks/s).

Lantern | *Python, FastAPI, LangChain, LangGraph, ChromaDB, Pydantic, pytest, Docker*

- Built a RAG service over uploaded PDFs that answers with page-level citations and declines when the documents lack the answer, using a ChromaDB ingestion pipeline and a LangChain chain with query rewriting and multi-turn memory.
- Added an opt-in LangGraph agent mode that decides when to search and which tools to call, with a tool-call cap and numbered citations; benchmarked it against the fixed pipeline at equal 100% accuracy but 2.6× the latency (3.6 s vs 1.4 s).
- Built a 28-case eval suite scoring retrieval and answers separately; used it to diagnose dropped output fields and raise accuracy from 86% to 100% with 92% citation precision.
- Injected models through FastAPI dependencies so 66 pytest tests run against fake LLMs and an in-memory vector store.

HomeHero | *Hackville 2026* | *SwiftUI, Spring Boot, Supabase, TCA*

- Built a full-stack household management app for shared tasks, shopping lists, expenses, and home data, in 24 hours.
- Developed the SwiftUI frontend using The Composable Architecture and async/await to manage application state and asynchronous workflows.
- Implemented a Spring Boot REST API for authentication, task operations, and multi-user data synchronization.

Technical Skills

Languages: Python, Java, TypeScript/JavaScript, Go, C, SQL, Swift
Frameworks: FastAPI, Spring Boot, React, Node.js, SwiftUI
Databases: PostgreSQL, Redis, Supabase, Firebase
AI & LLM: LangChain, LangGraph, ChromaDB, RAG
Cloud & DevOps: Git, Docker, Kubernetes, Azure, CI/CD
Libraries & Tools: pytest, pandas, scikit-learn